Technical report — PoesIA on Apertus

Verified 2026-10-13. Numbers are measured; sections marked *(to do)* are user-provided submission assets.

- **Track:** Track 2B — PoesIA on Apertus (Own Project)
- **Event:** Online
- **Team:** PoesIA — Angel Márquez Aguilar (solo)
- **Demo:** *(video link)*

1. Summary

People write when something moves them, and many would like to shape it into a poem, but form is hard to learn and AI writing tools take the words away from them. PoesIA is a local page where the person writes every line; Apertus v1.5 8B proposes lines only on request, and a deterministic engine checks syllables, stress and rhyme against the chosen form and explains each problem in plain language, in Spanish, English, Italian, German and French. A safety screen runs before any shaping. On a seeded benchmark of sonnets (18 per language) Apertus behind the engine writes 87–90% (en) and 87% (it) of lines in exact metre on one 8 GB consumer GPU with no network. Spanish: 50–58% by the corrected counter of 13 Oct, not the 83–85% first reported (§5, §6).

2. Architecture

```
browser ----> PoesIA page (FastAPI, port 8000) ----> scansion engine (deterministic)
                  |                                +-> safety screen (phrases + model question)
                  +---> OpenAI-compatible API ---> Ollama ---> Apertus v1.5 8B (GGUF Q4_K_M)
```

- **Page** (`poesia.webapp`): one language per session (es, en, it, de, fr) and four steps (what moved you → the form → write → listen and keep). Every line is a text field the author fills, scanned as they type (`/api/scan`) and shown as syllables ($la \cdot LU \cdot na_i \cdot lu \cdot MI \cdot na$). "Ideas" first offers common words that rhyme with the line's partner, from offline word lists, each with why it fits (`/api/words`); whole lines from Apertus come only on a second click (`/api/propose`), ranked by the scan, not by model likelihood, and shown greyed until used or edited. A whole draft comes only on request, as a starting point (a secondary link); its lines stay marked as drafts until edited, and "next" on an empty poem does not write it. Each line records its origin; "Keep" saves text, a printable page or a file to reopen, with the statement "N of M lines worded by the author". Drafts live in the tab.
- **Engine** (`poesia` package, five languages): metrical syllable count with synalepha, mute-e and final-stress rules, stress pattern, rhyme key by sound; explanations from `poesia.teaching`. Forms on the page: Spanish *soneto*, English Shakespearean sonnet, Italian *sonetto*, German *Sonett*, French sonnet in alexandrines, haiku, free verse.
- **Read-back**: Piper, offline, five stock voices with clear licences (es and de CC0, en public domain, it and fr CC BY 4.0, from rhasspy/piper-voices), labelled "synthetic voice"; no cloning.
- **Linking**: 3,000 poems each for es and en, 609 it, 676 de, 547 fr (Project Gutenberg books fetched for this purpose), all public domain or CC BY (credits in `data/linking/README.md`), matched by `bge-m3` embeddings on CPU, offered after writing.
- **Model**: Apertus v1.5 8B text backbone, served by Ollama 0.34.4.

Target architecture

a) On-premise and b) air-gapped. Everything runs on one machine, bound to `127.0.0.1`. `run.sh` / `run.ps1` (or plain `docker compose up`) start Ollama and the page; on first run the page downloads and prepares the models itself and shows the progress, and says when the model runs on the processor. For a machine with no network, `bundle.sh` packs both images and the prepared models (9.6 GB, with checksums) and `load.sh` / `load.ps1` restore them; tested by deleting images and models, restoring, and running on a Docker network with no route out (page, scan, safety, read-back, linking and suggestions all answered), and on Windows PowerShell 5.1. Without Docker, `run-native.sh` / `.ps1` start the page with only the Ollama app and uv installed (no compiler, no git).

c) Swiss sovereign cloud. The same page runs unchanged against the CSCS LLM Inference API (`LLM_BASE_URL=https://api.inference.cscs.ch/v1`, with the operator's own key): a three-line suggestion took 0.73 s on Apertus v1.5 8B and 0.99 s on the 70B (median of 10). CSCS states that it does not record prompts

or responses and that data stays in its infrastructure in Switzerland: an option for a canton or hospital without GPUs of its own.

3. Use of Apertus

- **Model:** `andreasmartin/apertus-v1.5-8b-text` (the official `swiss-ai/Apertus-v1.5-8B` with its image and audio towers removed), GGUF Q4_K_M (`Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, 4.8 GB; our own conversion with llama.cpp `4df29be` gave the same size to within 96 bytes).
- **How it is used:** inference only. (1) Line proposals: the engine builds a prompt with the poem so far, the syllable target and the rhyme to match; (2) line repair; (3) one yes/no safety question.
- **Where it runs:** local Ollama, 5.4 GB of GPU memory on an RTX 3070 (8 GB). Any OpenAI-compatible Apertus endpoint can replace it (`LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`).
- **Prompting detail that mattered:** with a rhyme word bank in the prompt, Apertus answered with the rhyme word alone ("more", "floor") in 6 of 6 seeded tries; a one-sentence system message ("…exactly one complete line of poetry… never a single word") brought that to 0 of 6. The Ollama Modelfile uses Apertus's chat format with deliberation disabled.

4. Data

- **At runtime:** none. The person's reflection and lines go only to the local model and are not stored or logged.
- **Safety test set** (`data/safety` in the engine repository; dataset card there): 60 synthetic reflections, 12 per language (es, en, de, fr, it): 20 ordinary, 20 sad but safe (grief, loneliness, autumn), 20 with a risk signal. No real person's text. *(to do: publish on Hugging Face with the organisers' template.)*
- **Evaluation gold** for the syllable counter: ADSO, 100 Spanish poems scanned by experts (CC BY-NC 4.0), used only to measure the counter.

5. Evaluation

Metre and rhyme. 6 themes × 3 seeds per language; Spanish *soneto* (11 syllables, ABBA ABBA CDC DCD), English Shakespearean sonnet (10, ABAB CDCD EFEF GG). Per line: syllables off target, share of lines in exact metre, share of rhyme pairs that hold. Same harness and seeds for every row; all rows scored by the same engine version.

Setup	es off/line	es metre	es rhyme	en off/line	en metre	en rhyme
Qwen2.5-1.5B + July 2026 adapter	1.23	31%	3%	0.35	73%	11%
Qwen3-4B, plain	0.70	45%	30%	0.69	56%	72%
Qwen3-4B + corpus adapter	0.44	64%	24%	0.46	75%	40%
Apertus 8B, raw prompt (transformers)	1.19	36%	41%	0.73	77%	83%
Apertus 8B, Ollama, line system message	0.78	52%	**44%**	**0.29**	**89%**	83%
same, plus dropping one- and two-word candidates	0.63	58%	33%	0.37	87%	86%

Setup	es off/line	es metre	es rhyme	en off/line	en metre	en rhyme
same, plus re-asking without the word bank when a batch is all bare words	0.63	50%	36%	**0.10**	**90%**	**88%**

Spanish columns: the stored poems rescanned on 13 Oct with the corrected counter (§6); first reported as 0.63/57%, 0.52/62%, 0.38/71%, 0.83/60%, 0.42/82%, 0.28/85%, 0.21/83%. The old counter also chose the lines in the loop, so a rerun with the corrected one in the loop is the fair figure.

Italian (Petrarchan *sonetto*, 11 syllables, ABBA ABBA CDE CDE; same harness, 6 themes × 3 seeds, Apertus via Ollama with the retry): **0.16 off per line, 87% of lines in metre, 43% of rhyme pairs**.

The last three rows differ by about their seed noise in metre and Spanish rhyme (single poems range from 0% to 100% rhyme), so we report all three. What the last change clearly did: lines that were only a rhyme word went from 9 (es) and 5 (en) to none. The benchmark's Spanish rhyme word bank comes from an online service (Datamuse); the deployed page sends none, so it runs offline.

Model size and precision (CSCS API for full precision and the 70B; llama.cpp on the RTX 3070 and again on a Hugging Face Jobs L4 for the local files; 36 seeded sonnets per language and setup, 72 for full precision; share of lines in exact metre / rhyme pairs):

Apertus v1.5	es	en	it
8B full precision (CSCS)	52–56%* / 39%	92.2% / 83%	89.3% / 53%
70B full precision (CSCS)	44–45%* / 46%	95.8% / 83%	82.1% / 48%
8B 6-bit Q6_K (3070; L4)	52–54%*; n/r / 41; 48%	94.2; 92.1% / 82; 89%	92.3; 91.3% / 61; 52%
8B 4-bit Q4_K_M, shipped (3070; L4)	50–55%*; n/r / 40; 38%	91.5; 90.5% / 88; 85%	84.7; 86.5% / 44; 34%

* Spanish metre rescanned with the corrected counter, range over the stored seeded runs (first reported 84.5%, 77.2%, 89.5% and 85.9%); n/r: the L4 runs kept no poems here, not rescanned.

Behind the engine the 70B is not better (Italian metre lower by about three standard errors; Spanish lower by the old counter and by the corrected one), so the entry stays on the 8B. The 4-bit file loses Italian metre and rhyme on both machines; 6-bit matches full precision and fits an 8 GB card (7.2 GB). Metre replicates across machines within about two points; rhyme moves by up to ten points between identical runs, and seeds do not reproduce single sonnets on any server.

Safety screen. On the author's own 60 items (20 risk, 40 safe) the screen reached 20/20 with no false alarms, but phrase list and items were written by the same hand. An independent set (`data/safety/generated/`) was therefore written by Apertus 70B on CSCS: 1,043 short reflections in five languages and seven categories, including idioms about dying, past distress and indirect warning signs, each then classified by the same model with a different question; 724 items where writer and judge agree. On them the screen caught only **78 of 280 risk texts (28%)**, missing passive death wishes ("better not to wake up") and indirect signs. The model question and phrase list were widened on that set and tested on a second, held-out set written before the change (708 agreed items):

Held-out set (708 items)	before, full precision	after, full precision	after, 6-bit Q6_K	after, 4-bit Q4_K_M (shipped)
risk texts caught	92 / 271 (34%)	**125 / 271 (46%)**	**129 / 271 (48%)**	109 / 271 (40%)

Held-out set (708 items)	before, full precision	after, full precision	after, 6-bit Q6_K	after, 4-bit Q4_K_M (shipped)
plain death wishes / indirect signs	52 / 104, 3 / 60	75 / 104, 17 / 60	76 / 104, 19 / 60	66 / 104, 14 / 60
false alarms on safe texts	6 / 437	**4 / 437**	4 / 437	2 / 437

Full precision ran on CSCS, the two GGUF files on a Hugging Face Jobs L4 (llama.cpp). The model question alone on the held-out set: 6-bit 111/271, 4-bit 84/271; on the 60 items 6-bit 20/20, 4-bit 18/20 (13/20 with the old wording). The 4-bit file loses about a sixth of the screen's catches, a second reason, after Italian metre, to prefer 6-bit. The screen is a safety net that still misses about half of these texts, not a guarantee; the pause itself says PoesIA is a writing tool, not a counselling service.

What the pause can and cannot claim (literature check, over 100 sources, engine repository `docs/research/reports/`). *Supported:* people report two to four times more suicidal thoughts when answers are anonymous and carry no consequences. PoesIA runs locally and reports nothing, which keeps it on that side. *Not supported:* that writing itself, or poetry, makes people more honest than a conversation with a clinician (untested for poetry; written and spoken disclosure were equal in a 94-study meta-analysis), or that the pause detects risk better than clinicians. No screen, clinical or automated, predicts suicide well at real base rates. *Not supported either:* treating anger or a wish for revenge as danger. Such thoughts are common and rarely lead to violence; threat assessment looks for stated intent, a plan or final-act language. The pause is therefore a safety net, not detection or screening for risk; wording that claimed detection would also make it a medical device under the EU MDR and Swiss MedDO.

Speed (same `make run` stack):

	RTX 3070 8 GB	CPU only (Ryzen 9 5900X, Docker Desktop)
rhyme words (word lists, no model)	at most 0.6 s first per language, then instant	same
a suggestion (6 candidates generated, best 3 shown)	1.4–5 s	13–15 s
safety screen	<1 s	2.0 s
linking (embed + search)	3.1 s	3.2 s
first suggestion after `make run` (model preloaded)	1.9 s	—

CPU-only memory with everything loaded: 7.3 GB. A fully automatic sonnet (the benchmark loop) takes ~35 s on the GPU. Energy, GPU board only (`nvidia-smi` once a second over 18 Italian sonnets): ~160 W while generating, **1.7 Wh per automatic sonnet**; a single on-request suggestion (~2 s) is under 0.1 Wh.

Accessibility, responsive layout, and user journeys (`ux-audit`, our open browser-driven audit library, and Playwright):

- **WCAG 2.2 AA Conformance:** all page controls pass mechanically decidable checks (accessible names, contrast, touch targets $\geq$ 24px, single `h1`, announced error containers);
- **Responsive Layout:** zero horizontal overflow across phone (390px), tablet (768px), and desktop (1024px+); zero controls below the iOS Safari 16px auto-zoom threshold;
- **Keyboard Navigation:** keyboard users reach and initiate the writing workflow strictly via `Tab` and `Enter` without requiring pointer input;
- **Persistence:** reflection, theme, and written lines survive page reloads and tab interruptions via `sessionStorage`;
- **End-to-End User Journeys** (automated in `tests/test_webui_ux_journey.py`):
  1. *An empty poem is not written for the person:* Spanish sonnet, theme and reflection, then "next" with no lines: the page stays on Step 3 and says how to start; no line is drafted unasked (cost: 4 interactions, 0 failures, 2.9 s);

2. *Thoughts per stanza, then a draft as a starting point:* optional thoughts in the 4 stanza fields, the secondary link *«Borrador de todos los versos, como punto de partida»*, 14 lines that stay marked as drafts until edited, then Step 4 and read-back by Piper (cost: 9 interactions, 0 failures, 20.9 s). Both run against the deployed page (Apertus v1.5 8B on CSCS), 2026-10-13.

- **Defects found and fixed during the audits:**
  - Revealing the next stanza redrew the page 350 ms after typing, stealing focus from fast typing; fixed by appending newly revealed stanzas rather than redrawing;
  - Changing the form silently cleared the lines; now prompts for confirmation;
  - Apertus draft prompt truncation stopping at 8 lines (the two quartets); resolved with an explicit 1-to-14 numbered verse template and resilient single-line fallback;
  - Language selection click dropped during the first 50 ms before `/api/forms` resolved; fixed by attaching click listeners immediately on DOM ready.

Model server. Same GGUF and card, through the page's own client: llama.cpp's server (b11459) scores the same as Ollama within run-to-run noise (es 53% vs 50% metre by the corrected counter, 84% vs 84% by the old, en 92% vs 89%, it 85% vs 83%), answers a three-line suggestion in 0.63 s vs 1.05 s, and its image is 2.59 GB (0.31 GB CPU-only) vs 3.75 GB. We keep Ollama for this entry (switching would cost the first-run progress screen and its tests) and plan the switch after it.

Linking: for "I saw a tree losing its leaves and thought of my father, who died in March" the page offers Dickinson, Barrett Browning and Millay ("Listen, children: / Your father is dead."); for the Spanish version, Rosalía de Castro ("Moría el sol, y las marchitas hojas").

Image and audio (official multimodal weights, 4-bit, same card): an image described in 2.2 s ("cats, remote controls, pink couch", correct) and a Spanish voice clip transcribed in 1.5 s with one word wrong (*parte* for *padre*); 7.5 GB of 8 GB reserved, so it is not in the default deployment.

People: *(to do: 2–3 tester sessions — did you finish, does it feel like yours, what got in the way.)*

6. Limitations

- Until 13 Oct the Spanish counter joined vowels after a final -n/-s ("los ojos" as 4) and agreed exactly with expert scansion on 59% of ADSO lines; the corrected counter (no synalepha after a consonant, licences as a set of allowed counts) agrees on 93.5% (99.6% within the licensed set). Every Spanish metre figure was first measured with the old one; §5 shows them rescanned.
- Rhyme is checked on the last word only (consonant rhyme); assonance and internal rhyme are not scored.
- German and French (added 9 Oct): syllables agree with expert annotation on 99.1% of 3,507 German lines (word stress on a stressed position 94.7%) and 95.9% of 72,412 French verses (alexandrines 97.5%); the rules were tuned on these same sets. French rhyme by sound agrees with eSpeak NG on 98.4% of frequent line endings. Behind the engine (36 sonnets each) the 8B writes 80–85% (de) and 86–93% (fr) of lines in exact metre (4-bit to full precision); the 70B 95% (de) and 85% (fr). Romansh has no scansion. Italian counting agrees exactly with expert scansion on 79.5% of 10,001 annotated endecasillabi (Dante, Petrarca; mean error 0.23); dialefe is its main miss. Word stress comes from Wiktionary for 44,272 words (unmarked *sdruccioli* such as *crescita*, *tavola*): agreement with the experts' stresses rose from 80.2% to 82.3% (precision 90.0% to 92.3%); an attempt to take stress from Apertus 70B failed (about half wrong).
- The 8B slips into English: 10–22% of benchmark lines in Italian, German and French (under 2% in Spanish), at full precision as in the 4- and 6-bit files; the 70B 1–2%. Metre scores count those lines too. The page ranks proposals in another language last, and flags any line that looks like another language.
- Seeded runs do not reproduce exactly on Ollama (4 of 7 Italian sonnets differed between two runs with the same seeds), so benchmark differences of a few points are noise.
- Italian rhyme-word suggestions leave out longer words whose stress the engine cannot be sure of, so fewer are offered than in Spanish or English.
- Ollama 0.40.0 on Windows 11 25H2 with Developer Mode cannot read models it pulls ("untrusted mount point"); there the no-Docker start fails with that message, and the Docker start works.
- The safety screen caught 46% of risk texts on the held-out model-written set. Those texts were written and labelled by a model, not reviewed by a person, and say nothing about slang, irony or Swiss German dialect. PoesIA makes no therapeutic claim.

7. Reproducibility

- Hardware: RTX 3070 8 GB, Ryzen 9 5900X, Windows 11 + WSL2, Docker Desktop 29.8.
- Engine: `OomAngel/poesia` at the commit pinned in `src/Dockerfile` (`POESIA_REF`).
- Model: GGUF as in section 3; Ollama 0.34.4.
- Benchmark: `scripts/evaluate_adapter_mlflow.py --ollama-model <name> --samples 3 --seed 0` (or `--openai-compat` for any endpoint, as the page calls it) in the engine repository; server comparison: `reports/eval_2026-10/p2-server-comparison.json`; safety: `scripts/evaluate_safety_screen.py [--data …] [--agreed-only]`; test-set writer: `scripts/generate_safety_set.py`; quantisation on Hugging Face Jobs: `scripts/hf_jobs/quant_eval.sh`; every CSCS batch with its cost and result: `docs/hack_apertus/CSCS_USAGE.md` (89,973 requests, CHF 0.88 in three days).

8. Before and during the hackathon

The engine predates the hackathon: PoesIA's scansion, rhyme and generation loop were built from July 2026 (first commit 2026-07-26) and are open source. Built during the hackathon (1–16 Oct): this repository and its Docker deployment; the writing page; the safety screen and its test set; Apertus integration (runtime, chat format, the system-message fix, the OpenAI-compatible client); the Apertus evaluation; the guided page, rhyme-word suggestions and Italian scansion view; German and French (scanners, page, voices, linking); first-run setup, the offline bundle and the no-Docker start; the Italian linking set; the CSCS and Hugging Face Jobs evaluations, the model-written safety sets and the retuned screen; and fixes found on the way (English rhyme keys, model selection, offline NLTK, focus while typing). The engine repository's history shows the dates.

9. Next steps

German and French scansion and page languages; moving the model server to llama.cpp; short phrase suggestions between words and whole lines; fine-tuning Apertus on public-domain verse in the five languages, kept only if it beats plain Apertus on this benchmark; the photo and spoken reflection on Apertus's own image and audio input, on a card with room for it.

Acknowledgements

The Hack Apertus organisers passed on a CSCS inference key after a late request and gave Hugging Face credits; the size, precision and safety findings above come from them. Thanks to CSCS for the inference service, to the Swiss AI Initiative for Apertus, and to Andreas Martin for the text-only model.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

(to do: from the engine repository's `docs/hack_apertus/refs.bib`, once validated.)